Object Classification with Small Data: A Case Study on Rat Images

Nathaniel McComas
College of Natural and Health Sciences
University of Hawai'i at Hilo
Hilo, Hawai'i, USA
nmccomas@hawaii.edu

***Abstract*—One of the central challenges in machine learning and other data science fields is obtaining enough labeled data to train a model effectively. This challenge is especially pronounced in specialized domains, where collecting and annotating data can be expensive and time consuming. This paper studies object classification with small-data constraints through a case study on classifying images of pet rats. We compare transfer-learned CNNs with different augmentation techniques, learning rates, and backbone-freezing schedules to examine how these choices affect generalization to unseen data within this dataset. The resulting discussion offers insights into which design choices are important to consider in the small-data regime.**

***Index Terms*—small data, transfer learning, convolutional neural networks, object classification, model architecture, data augmentation**

I. Introduction

Object classification remains one of the core tasks in computer vision, and deep convolutional models have achieved strong performance on large-scale benchmarks such as ImageNet [1], [2]. However, successful image classification systems are often built on datasets that are far larger and more diverse than what is available in many real-world settings. In specialized domains, obtaining enough labeled data to train a robust model can be difficult, expensive, or both. Small datasets also amplify practical problems such as overfitting and poor generalization to new environments or individuals [3].

This paper studies object classification with limited data through a case study on pet rat images. Rather than determining only which architecture is strongest in general, we examine which design choices produce the best generalization to unseen data. We focus on transfer learning [4], augmentation [5], model family and size [6]–[8], and simple training-time choices such as learning rate and how long to keep the feature extractor/backbone frozen. The goal is not to identify a single best model and/or set of design choices, but rather to understand how these factors interact and which factor is most important for improving generalization during testing in the small-data regime.

A. Related Work

There has been a significant amount of research on techniques for improving model performance with small datasets. Two of the most common approaches are transfer learning, in which a model pre-trained on a large dataset is fine-tuned on a smaller target dataset [4], and data augmentation, which creates additional training examples by applying label-preserving transformations such as cropping, flipping, rotating, and color jittering [5]. Often, these techniques are combined so that a model starts from broadly useful visual features while also seeing more variation during training [4], [5].

B. Paper Organization

The rest of this paper is organized as follows: In Section II, we provide further background on object classification with small datasets and briefly discuss our dataset. In Section III, we describe our methods, including the experimental setup and implementation details. In Section IV, we present the results of our experiments, including performance metrics and comparisons between different techniques. In Section V, we discuss the implications of our results, including the strengths and limitations of our approach. Finally, in Section VI, we summarize the main takeaways from our work and potential directions for future work.

II. Background

There are several challenges associated with object classification using small datasets. These include limited data availability, elevated risk of overfitting, and difficulty generalizing to unseen examples. In this section, we summarize the concepts most relevant to our study: transfer learning, data augmentation, and architecture selection. We also provide details about the dataset we will be using in our experiments, including its structure and potential limitations.

A. Techniques for Small Data

When working with small datasets, some of the most common techniques for improving performance and generalization include transfer learning, data augmentation, and careful selection of model architecture and training parameters.

1) Transfer Learning: As mentioned in the introduction, transfer learning uses a model pre-trained on a large source dataset and adapts it to a smaller target dataset [4]. This approach allows the model to reuse previously learned patterns, which is especially helpful when the target dataset is too small to support strong learning from scratch. For example, models pre-trained on ImageNet learn generic features such as edges,

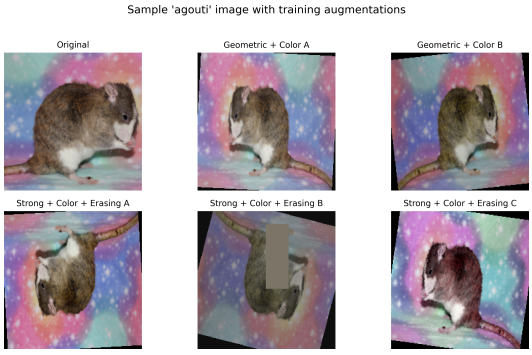

Fig. 1. Example of training-time augmentation applied to one rat image from our dataset. The top-left panel shows the original image after resizing and center cropping for display, while the remaining panels show representative outputs produced by two different augmentation pipelines. Thanks to Little Paws Rattery for allowing us to use this image.

textures, and patterns that transfer well across many visual tasks [1]. By fine-tuning these models on our rat dataset, we are essentially jump-starting the learning process with many useful patterns already in place, which can help improve generalization to unseen data.

Without transfer learning, training a deep network from scratch on a small dataset would be much more likely to lead to severe overfitting [3]. For that reason, pre-trained initialization is a natural default for this project, since it is extremely unlikely that training from scratch would yield any meaningful results.

2) Data Augmentation: Data augmentation is a technique used to artificially increase the effective size of a training dataset by applying label-preserving transformations to existing data. This can help improve model performance by providing more variation during training and reducing overfitting. In small datasets, a model may otherwise learn to associate specific poses, orientations, environments, or lighting conditions with the class labels, which can lead to poor generalization. Applying transformations such as random cropping, flipping, rotating, and color jittering can help the model learn more robust features that are less tied to those conditions.

Beyond these basic transformations, more advanced augmentation strategies such as random erasing [9] and composition-based methods such as CutMix [10] can further regularize training, although this work only includes a limited exploration of these techniques [5]. Refer to figure 1 for examples of what training-time augmentation may look like for one rat image from our dataset.

3) Model Architecture Considerations: When working with small datasets, the choice of model architecture can have a significant impact on performance. Models with too many parameters may be prone to overfitting, while models that are too simple may lack the capacity to learn meaningful patterns from the data [3]. Therefore, it is important to consider architectural complexity in relation to dataset size. In general, regularization methods such as dropout and/or weight decay can also help reduce overfitting when data are limited [11]. We will be using weight decay in our experiments, but we do not include dropout.

Besides model size, the specific architecture can also play a role. Convolutional neural networks (CNNs) are commonly used for image classification tasks due to their ability to learn spatial hierarchies of features [2], [12]. However, even within the realm of CNNs, there are many different architectures to choose from, such as ResNet [6], EfficientNet [8], and MobileNet [7], each with its own trade-offs in terms of complexity and performance. When working with small datasets, it is likely beneficial to carefully select an architecture that is well-suited to the task and the amount of data available.

B. Dataset

Our dataset consists of images of pet rats collected from breeder websites and public social-media pages, with permission granted for their use in this project. Specific breeder credits are provided in the Acknowledgment section. The dataset contains images from five coat-based classes:

1) Agouti
2) Black
3) Blue
4) Marten
5) Double Rex/Hairless

In total, the dataset contains 554 images organized under 129 nested folders. Most of these folders correspond to individual rats, however there is an assortment folder within each class for rats with only one or two images available. The class image counts are 136 Agouti, 122 Black, 113 Blue, 138 Marten, and 45 Double Rex/Hairless. Counting the nested folders used by the split procedure yields 34 Agouti folders, 17 Black, 26 Blue, 35 Marten, and 17 Double Rex/Hairless.

There are a number of important considerations regarding this dataset that could impact model performance and generalization.

1) Individuals and Environments: Most folders in the dataset correspond to a single identified rat, which lets us test generalization across individuals rather than only across images. In addition, each class contains an assortment folder that groups together rats from that class when only one or two images are available for a given individual instead of creating many separate sparse folders.

Among the five classes, there is an imbalance in the number of individuals represented, with some classes containing many more individuals than others. Furthermore, some classes simply have much fewer images overall, which could make it difficult for models to learn robust features for those classes. This imbalance could lead to models learning spurious correlations that are specific to the individuals or environments represented in the dataset, rather than learning generalizable features that are relevant to the class as a whole.

It is our hope that strong data augmentation and careful model architecture choices can help mitigate these issues and allow the models to learn more robust features that generalize well to new individuals and environments. However, these are

important limitations to keep in mind when interpreting the results of our experiments.

2) Juveniles: A very large portion of the rats are juveniles, since many of the pictures shared by participating breeders are of new litters. This could lead to models learning to identify juvenile features rather than features specific to the intended class, which could also limit generalization to new images of adult rats within the same class.

Again, we hope that strong data augmentation and careful model architecture choices can help mitigate this issue and allow the models to learn more robust features that generalize well to both juvenile and adult rats. However, this is another important limitation to keep in mind when interpreting the results of our experiments.

3) Note about Rat Genetics and Classification: It is important to note that the classification of rats into these 5 different classes is not a strictly scientific classification based on genetics, but rather a more informal classification based on physical appearance and traits. It is also important to note that many of these classes are not mutually exclusive, and there may be some overlap between them. For example, a rat could simultaneously be a blue, marten, and agouti. This is because these classes are based on physical traits that can be influenced by multiple genetic factors, and there is not always a clear-cut distinction between them. Therefore, it is important to keep in mind that the classification of rats into these classes is somewhat subjective.

In order to determine the class labels for our dataset, we chose our own hierarchy of traits to use for classification, which is based on the visible characteristics of the rats in the images. We acknowledge that this classification is not perfect, but it provides a consistent framework for labeling our dataset and conducting our experiments. In order, we used the following hierarchy of traits for classification:

1) Double Rex/Hairless: If the rat has a double rex coat OR it is hairless, it is classified as a Double Rex/Hairless. Although these are not necessarily the same trait, they are grouped together for classification purposes. They are both characterized by partial or complete absence of fur. This trait is very easy to identify and classify based on visual characteristics, making it a clear choice for the first level of our classification hierarchy.
2) Marten: If the rat has a marten coat, it is classified as a Marten. This is characterized by dark coloration on the extremities (face, feet, tail) which fades into a lighter color on the body. The most distinctive feature is the smooth gradient from dark to light. This trait is relatively easy to identify and classify based on visual characteristics, making it a clear choice for the second level of our classification hierarchy. It is noteworthy that Martens more often have red eyes than other classes, which could be a confounding factor for classification.
3) Blue: If the rat has a blue coat, it is classified as a Blue. Despite their name, they are often characterized by a partial or complete dark-gray coat, rarely with a bluish tint. This trait is relatively easy to identify and classify based on visual characteristics, making it a clear choice for the third level of our classification hierarchy. They are somewhat visually similar to Martens, since both of them have a grayish appearance, although Martens have a distinctive gradient and may not always be gray. As Martens are more visually interesting, they take precedence over Blues in our classification hierarchy, which means that if a rat has both traits, it is classified as a Marten rather than a Blue.
4) Agouti: If the rat has an agouti coat and no other higher precedence traits (as defined above), it is classified as an Agouti. This is characterized by multiple hair bands of different colors on each individual hair, which creates a "wild-type" coat color that is often brown or gray with a speckled appearance. This trait, although visually distinct, can be more difficult to identify and classify than the other traits due to its variability and potential overlap with other classes (e.g., a rat could be both an Agouti and a Blue). We have decided to place it lower in the hierarchy somewhat arbitrarily, and we acknowledge that it could be placed higher or lower without a strong justification. However, we have chosen to place it above Black because the Agouti trait provides more specific information about the coat pattern and coloration than the Black trait alone, which is simply a solid black coat color.
5) Black: If the rat has a black coat and no other higher precedence traits (as defined above), it is classified as a Black. This is characterized by a solid black coat color on parts of or the entire body. This trait is relatively easy to identify, however it is placed last in the classification hierarchy because it is the least visually interesting trait, and it is very common. For example, a rat could be both a Black and a Marten, but the Marten trait would take precedence in our hierarchy because it provides more specific information about the coat pattern and coloration than the Black trait alone.

Refer to figure 2 for example images from each of the five classes in our dataset.

4) Dataset Splitting: For our experiments, we split the dataset into training, validation, and test sets. The training set is used for parameter updates. The validation set is used for early stopping and for comparing configurations during the sweep. Each set is disjoint, and the test set is only used for final evaluation after training is complete.

The split is identity-based rather than image-based: each rat folder is assigned wholly to one split. This design is intended to test whether a model can generalize to unseen rats rather than merely unseen photographs of previously seen rats. The target split ratios are 60% training, 15% validation, and 25% test.

Because the split operates on identity folders and uses best-effort allocation for small classes, the realized split is not exactly proportional at the image level. In our dataset, the resulting split contains 311 training images, 81 validation

Fig. 2. Example images from each of the five classes in our dataset. Thanks to Little Paws Rattery and Kolohe 'Iole Rattery for allowing us to use these images.

images, and 162 test images, corresponding to 77, 20, and 32 immediate nested folders respectively.

III. Methods

To investigate object classification with small datasets, we designed a controlled experiment around our dataset and varied only a small set of training factors at a time. All candidate models used ImageNet-pretrained weights [1], and each run replaced the final classification head to predict the five dataset classes. We then compared families, model sizes, learning rates, augmentation pipelines, and backbone-freezing schedules while keeping the rest of the training recipe constant.

A. Experimental Setup

Our sweep evaluates four convolutional-network families: ResNet, MobileNet, EfficientNet, and DenseNet [6]–[8]. For each family, we test a smaller and a medium-sized variant, giving eight architectures in total. Each architecture is trained with two learning rates (10^{-4} and 3×10^{-3}), two freezing schedules for the pre-trained backbone (freeze for the first 5 epochs or the first 15 epochs), and six augmentation pipelines ranging from minimal resizing and center cropping to strong geometric and color augmentation with random erasing [5]. This yields 192 experiment configurations overall.

Across all runs, the batch size is fixed at 16. Training uses AdamW [13] with weight decay of 10^{-4}, a cosine-annealing learning-rate schedule [14], a maximum of 40 epochs, and early stopping after 10 epochs without improvement in validation loss. During the frozen-backbone phase, only the classifier head is updated. Then, once the freeze period ends, the full network is fine-tuned. For each run, the best checkpoint is selected according to validation loss, and that trained model is then evaluated on both the validation and test splits.

B. Implementation Details

The project is implemented in PyTorch and Torchvision. Dataset loading is handled with `ImageFolder`, which assigns labels from the top-level class directories and recursively discovers images inside the nested identity folders.

Evaluation reports overall accuracy, macro-averaged precision, macro-averaged recall, and per-class counts of correct predictions, total examples, and predicted examples. These metrics are saved alongside the trained model weights for each run. Because the dataset is both small and somewhat imbalanced, we record macro-averaged metrics in addition to accuracy so that performance on minority classes is not obscured by stronger performance on the more represented classes.

1) Data Augmentation Pipelines: The six augmentation pipelines are as follows:

- Minimal: Resize to 224x224 and normalize with ImageNet mean and std.
- Basic Geometric: Minimal + random horizontal flip and random rotation of up to 10 degrees.
- Strong Geometric: Minimal + random horizontal/vertical flips, and random rotation of up to 25 degrees.
- Basic Geometric + Color: Basic Geometric + random color jittering (brightness, contrast, saturation up to 0.15, hue up to 0.05)
- Strong Geometric + Color: Strong Geometric + random color jittering (brightness, contrast, saturation up to 0.3, hue up to 0.1)
- Strong Geometric + Color + Random Erasing: Strong Geometric + Color + random erasing with probability 0.25, scale (0.02, 0.15), and ratio (0.3, 3.3)

These are all implemented using the `torchvision.transforms` module, and the parameters were chosen to provide a range of augmentation strengths while still producing visually plausible images. See Figure 1 for examples of the outputs produced by these pipelines.

2) Model Architectures with Size Variants: As mentioned, we evaluate four CNN families with two size variants each. However, "small" and "medium" are relative terms that depend on the specific architecture. Hence, we define the size variants for each family as follows:

- ResNet: ResNet-18 (small) and ResNet-34 (medium)
- MobileNet: MobileNetV3-Small (small) and MobileNetV2 (medium)
- EfficientNet: EfficientNet-B0 (small) and EfficientNet-B2 (medium)
- DenseNet: DenseNet-121 (small) and DenseNet-169 (medium)

In general, the "small" variants are the smallest models available within their respective families that are still commonly used, while the "medium" variants are slightly larger models that are still lightweight compared to the largest models in those families.

For future reference, by "model architecture" or "architecture" we may be referring to the family (e.g., ResNet) or the specific size variant (e.g., ResNet-18).

3) Splitting Details: The split procedure begins by parsing each image path relative to the dataset root and extracting its class folder and immediate identity folder. It then builds a mapping from class name to rat name to the list of sample

indices belonging to that rat. In other words, the split logic never operates on individual images directly; it operates on whole identity folders and then expands those folder assignments back into image indices.

For the experiments in this paper, the requested split ratios are 0.60, 0.15, and 0.25 for training, validation, and test respectively, with a fixed seed of 42. The procedure first normalizes the requested ratios so they sum to 1, then processes each class independently. Within each class, rat-folder names are sorted and then deterministically shuffled using a class-specific pseudorandom stream derived from the seed and class name. This means the split is reproducible across runs while still avoiding a bias from alphabetical folder order.

After shuffling, the procedure allocates how many identities from that class should go to each subset using a best-effort routine designed for small classes. The identities are distributed according to the requested ratios by combining floor-based allocation with a remainder rule that prioritizes larger unmet fractional targets. Once the rat folders assigned to each subset are known, all image indices from those folders are appended to the corresponding train, validation, or test index list.

IV. Results

This section reports the results of the full experiment sweep. In particular, this section compares model families, model sizes, augmentation strengths, and freezing schedules using validation and test accuracy, macro precision, and macro recall. This section also details per-class performance and discusses how performance varies across classes with different levels of representation in the dataset. Finally, this section examines which design choices were associated with stronger or weaker generalization within this experiment.

A. Per Class Performance

Likely the most interesting aspect of the results is how performance varies across classes within the dataset. Because the dataset is small and imbalanced with respect to the number of individuals and images per class, it is important to understand how well the models are able to learn robust features for each class and whether some classes are more difficult than others. This can provide insights into the strengths and weaknesses of the models and the dataset itself, as well as inform future efforts to improve performance on underrepresented classes. Refer to figure 3 for a summary of per-class precision and recall averaged across all models and training configurations.

The most notable trend is that the Blue class consistently shows the weakest performance across all models and training configurations, with average precision 0.43 and average recall 0.19. This suggests that the visual features of the Blue class may be more difficult for the models to capture and generalize from, which is almost certainly due to overlap with other classes. Indeed, the Blue class is often confused with the Marten class, and the per-class misclassification results show that Marten is overwhelmingly predicted for Blue images. This pattern is strong enough that the overall performance of models appears to be closely tied to their ability to learn

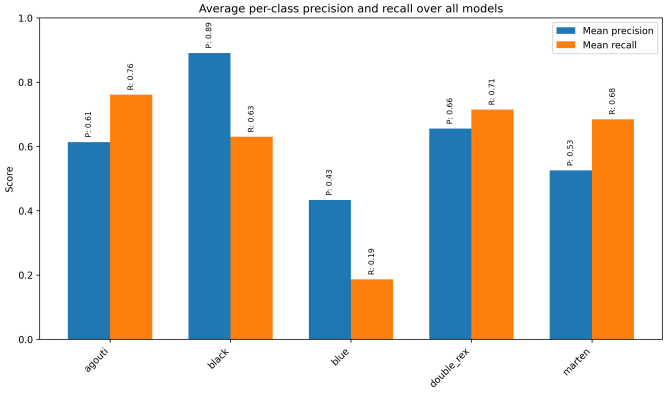

Fig. 3. Per-class precision and recall on the test set averaged across all models and training configurations. The Blue class shows the weakest precision and recall. The Agouti class shows the highest recall, and the Black class the highest precision. Despite the severe underrepresentation of the Double Rex/Hairless class, it has acceptable precision and recall, likely due to its distinctive visual features.

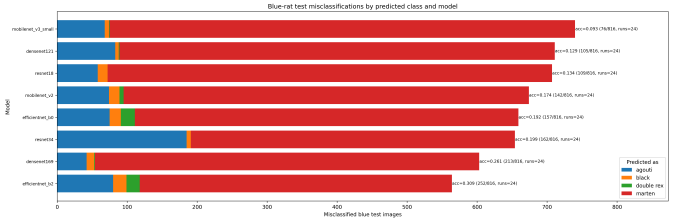

Fig. 4. Per-class misclassification for the Blue class across the 8 different model architectures. All of the Blue test images were run through the trained models, and the predicted classes were recorded and aggregated across each architecture. MobileNet v3 small only correctly classified 9.3% of the Blue test images, making it the weakest architecture for that class. In comparison, EfficientNet B2 correctly classified 30.9% of the Blue test images, making it the strongest architecture for that class.

features that separate Blue from Marten. See figure 4 for the per-class misclassification of the Blue class across the 8 different model architectures. MobileNet v3 small correctly classified only 9% of the Blue test images, making it the weakest architecture for that class. In comparison, EfficientNet B2 correctly classified 30% of the Blue test images, making it the strongest architecture for that class.

In contrast, Agouti shows the highest recall across all models and training configurations at 0.76, which suggests that the models more reliably recovered that class on this dataset. The Black class shows the highest precision at 0.89, which suggests that Black predictions were usually correct when made. However, Black recall is lower at 0.63, which suggests that the models still missed a substantial portion of Black examples or confused them with other classes. Taken together, these results indicate that some classes were easier for the models to separate than others in this experiment.

What is interesting is that the Double Rex/Hairless class, which is severely underrepresented in the dataset, still shows acceptable precision and recall at 0.66 and 0.71 respectively. This is likely because the visual features of that class are very distinctive, which makes it easier for the models to learn robust

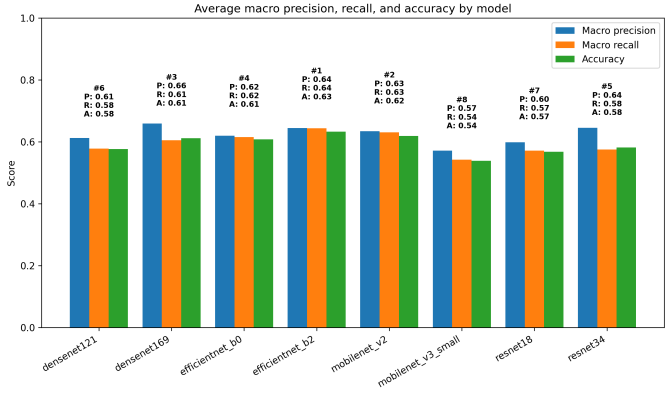

Fig. 5. Comparison of model families and sizes based on test accuracy, macro precision, and macro recall. Each bar represents the average performance across all training configurations for that model architecture.

features for that class despite the limited number of training examples.

B. Model Family and Size Comparisons

This subsection compares how different architectures performed in this sweep and whether smaller or larger models tended to perform better in this small-data setting.

It first helps to directly compare the overall performance of the different model families and sizes. See figure 5 for a summary of test accuracy, macro precision, and macro recall averaged across all training configurations for each model architecture. Overall, EfficientNet B2 shows the strongest average performance, with macro precision 0.64, macro recall 0.64, and accuracy 0.63, while MobileNet v3 small shows the weakest, with macro precision 0.57, macro recall 0.54, and accuracy 0.54. This gap suggests that architecture choice had a meaningful effect on performance in this small-data setting, although the variability within each model indicates that other design choices such as augmentation strength and freezing schedule still played a role in generalization.

What is important to note is that the overall performance of the different model architectures appears to be closely related to their ability to correctly classify the Blue class, which is the weakest class overall. In this sweep, architectures that handled the Blue class better also tended to perform better overall. For example, EfficientNet B2, which shows the strongest overall performance, correctly classifies about 30% of the Blue test images, while MobileNet v3 small, which shows the weakest overall performance, correctly classifies only about 9%.

Furthermore, there is some evidence that larger models within the same family tend to outperform their smaller counterparts on average in this sweep. For example, EfficientNet B2 slightly outperforms EfficientNet B0 (0.64/0.64/0.63 vs. 0.62/0.62/0.61 for macro precision, macro recall, and accuracy), and MobileNet v2 outperforms MobileNet v3 small (0.63/0.63/0.62 vs. 0.57/0.54/0.54). However, there is still a lot of variability across training configurations for each architecture, and some smaller models perform better than

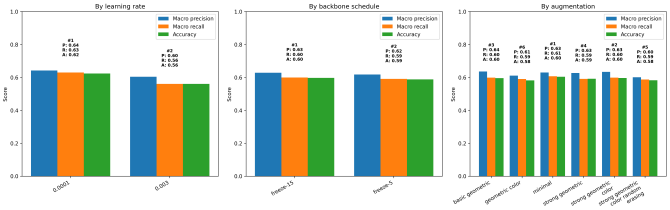

Fig. 6. Comparison of each training strategy (learning rate, augmentation strength, freezing schedule) based on test accuracy, macro precision, and macro recall. Each bar represents the average performance across all model architectures for that training strategy, with each of the other strategies varied.

some larger models on certain configurations. At most, these results suggest a modest within-family advantage for the larger variants under this training setup rather than a general rule for small-data problems.

C. Overall Training Configuration Comparisons

This subsection compares how each of the design choices (learning rate, augmentation strength, freezing schedule) affected performance in this experiment.

See figure 6 for a summary of test accuracy, macro precision, and macro recall averaged across all model architectures for each training strategy. Augmentation strength does not show a clear trend in this sweep, and the strongest augmentation pipeline on average is the minimal pipeline, with macro precision 0.63, macro recall 0.61, and accuracy 0.60. The second-best augmentation strategy is strong geometric + color, which is close behind at 0.63, 0.60, and 0.60. Within this experiment, stronger augmentation therefore does not appear to provide a meaningful average gain. Similarly, the longer freezing schedule shows only a marginal increase in performance, with freeze-15 averaging 0.63 macro precision, 0.60 macro recall, and 0.60 accuracy, compared with 0.62, 0.59, and 0.59 for freeze-5. However, the learning rate shows a clearer trend, with the lower learning rate of 10^{-4} averaging 0.64 macro precision, 0.63 macro recall, and 0.62 accuracy, compared with 0.60, 0.56, and 0.56 for the higher learning rate of 3×10^{-3}. In many of the high-learning-rate runs, the loss also appeared unstable and sometimes jumped upward by roughly two to three orders of magnitude, which is consistent with the weaker average performance observed for that setting. In this study, the lower learning rate is the strongest and most consistent training-strategy effect among the three factors examined.

The raw averages in figure 6 may obscure architecture-specific interactions between the training strategies. That possibility is one reason to examine multi-variable interactions next rather than relying only on the pooled averages.

D. Interactions Between Up To Two Design Choices

This subsection examines pairwise interactions between design choices to see whether any combinations stand out within the experiment. Learning rate is not included because its overall trend is clearer than those of the other design

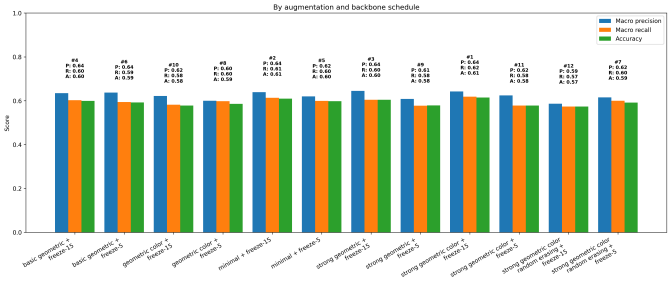

Fig. 7. Interaction between backbone freezing schedule and augmentation strength based on test accuracy, macro precision, and macro recall. Each set of bars represents the average performance across all model architectures for that combination of freezing schedule and augmentation strength, with the learning rate varied.

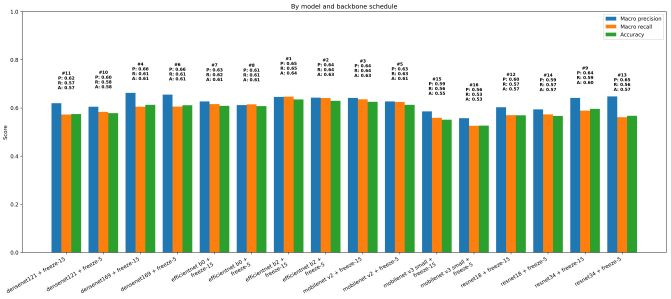

Fig. 8. Interaction between backbone freezing schedule and model architecture based on test accuracy, macro precision, and macro recall. Each set of bars represents the average performance across all training configurations for that combination of freezing schedule and model architecture, with the learning rate and augmentation strength varied.

choices. Interactions among three or more choices will not be considered because they would be harder to interpret and are likely susceptible to noise given the size of the dataset and the number of training configurations.

1) Interaction Between Backbone Freezing Schedule and Augmentation Strength: First, consider the interaction between backbone freezing schedule and augmentation strength. See figure 7 for a summary of test accuracy, macro precision, and macro recall averaged across all model architectures for each combination of freezing schedule and augmentation strength. There does not appear to be a clear trend in terms of which combinations are most effective, which suggests that any interaction between these two design choices is modest at best in this sweep. The combination of freezing for the first 15 epochs and the strong geometric + color augmentation pipeline shows the highest average performance, with macro precision 0.64, macro recall 0.62, and accuracy 0.61, while the combination of freezing for the first 15 epochs and the strong geometric + color + random erasing augmentation pipeline shows the lowest average performance, with 0.59, 0.57, and 0.57. Even so, the absolute differences remain fairly small.

2) Interaction Between Backbone Freezing Schedule and Model Architecture: It is also useful to examine the interaction between backbone freezing schedule and model architecture. See figure 8 for a summary of test accuracy, macro precision, and macro recall averaged across all training configurations for each combination of freezing schedule and model architecture. There does not appear to be a clear trend in terms of which combinations are most effective, which suggests that this interaction is also limited in this experiment. In fact, the two strongest combinations are both with EfficientNet B2, with one of the freezing schedules each, and the two weakest combinations are both with MobileNet v3 small, again with one of the freezing schedules each. In general, most models show similar performance across both freezing schedules, which suggests that the choice of freezing schedule was not a dominant factor for most architectures here.

Despite this, 7 out of 8 models show marginally stronger performance with the longer freezing schedule than with the shorter freezing schedule. That pattern provides some evidence

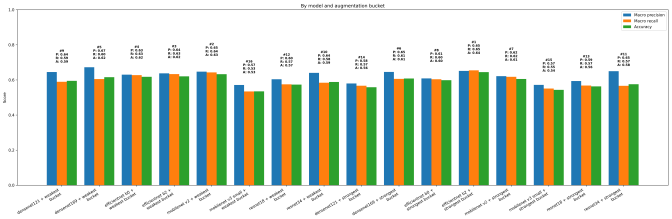

Fig. 9. Interaction between augmentation strength and model architecture based on test accuracy, macro precision, and macro recall. Each set of bars represents the average performance across all training configurations for that combination of augmentation strength and model architecture, with the learning rate and freezing schedule varied. The three weakest augmentation strategies (minimal, basic geometric, and strong geometric) are pooled together, as well as the three strongest augmentation strategies (basic geometric + color, strong geometric + color, and strong geometric + color + random erasing).

that longer freezing may be somewhat helpful across a range of architectures under this setup.

3) Interaction Between Augmentation Strength and Model Architecture: Finally, consider the interaction between augmentation strength and model architecture. See figure 9 for a summary of test accuracy, macro precision, and macro recall averaged across all training configurations for each combination of augmentation strength and model architecture. Note that the three weakest augmentation strategies are pooled together, as are the three strongest augmentation strategies, so that we are comparing the extremes. There does not appear to be a clear trend in terms of which combinations are most effective, which suggests that any interaction between augmentation strength and architecture is also minimal in this experiment. In general, most models show similar performance across different augmentation strengths, so augmentation strength does not appear to be a dominant factor for most architectures in this sweep.

V. Discussion

The discussion focuses on which design choices were associated with stronger or weaker performance in this experiment and on how these observations should be interpreted. Particular attention is given to whether stronger augmentation

improved robustness across individuals, whether lighter-weight architectures outperformed larger ones on this dataset, and how much performance varied across classes with different levels of representation.

A. Benefits of Design Choices

Overall, the results suggest that using a lower learning rate was associated with more stable generalization in this experiment. Across all runs, the lower learning rate averaged 0.64 macro precision, 0.63 macro recall, and 0.62 accuracy, compared with 0.60, 0.56, and 0.56 for the higher learning rate. In addition, many of the runs with the higher learning rate showed unstable loss behavior, with the loss sometimes jumping by 2 to 3 orders of magnitude. Within the scope of this study, that gap in both metrics and training stability makes learning rate the clearest training-strategy effect among the factors examined.

However, the results do not show a clear trend in terms of which augmentation strategies were most effective. In fact, the minimal augmentation pipeline slightly outperformed the next-best strong geometric + color pipeline, with 0.63/0.61/0.60 versus 0.63/0.60/0.60 for macro precision, macro recall, and accuracy. In this dataset, increasing augmentation strength therefore did not produce a meaningful average gain and may sometimes have been counterproductive. Similarly, although the longer freezing schedule shows a marginal increase in performance, the difference is small: freeze-15 averaged 0.63/0.60/0.60, compared with 0.62/0.59/0.59 for freeze-5. At most, this provides weak evidence that longer freezing may help slightly under this setup.

It is important to consider the limitations of this study when interpreting the results. Further work may include expanding the dataset, incorporating more formal genetic labels, or even using an entirely different dataset to see if the trends observed here hold in other contexts. Additionally, it may be worth exploring other design choices such as cross-validation, class-balanced losses, or self-supervised pretraining to see if they can further improve performance in the small-data regime.

B. Choice of Model Architecture

The results suggest that model architecture had a larger association with performance than the training-strategy differences examined here, though that conclusion should be limited to these experiments. The differences between architectures are visibly larger than the differences between augmentation or freezing strategies, and larger models within the same family often performed somewhat better. However, there is still substantial variability across training configurations for each architecture, and some models that perform worse on average can outperform stronger models on particular configurations (see figures 7, 9, 8). These results support treating architecture choice as an important variable in this experiment, not as a universally dominant factor in all small-data settings.

Although larger models within the same family often perform better than their smaller counterparts, larger models by raw parameter count do not necessarily perform better than smaller models from a different family. For example, ResNet-34 is the model with the largest parameter count, but it places fifth overall in terms of performance, while MobileNet v2 is the model with the second smallest parameter count, but it places second overall in terms of performance (see figure 10). Within this experiment, that pattern suggests that model family may matter at least as much as raw size, and perhaps more. It would be premature, however, to treat that as a general statement about architecture selection beyond this dataset and training recipe.

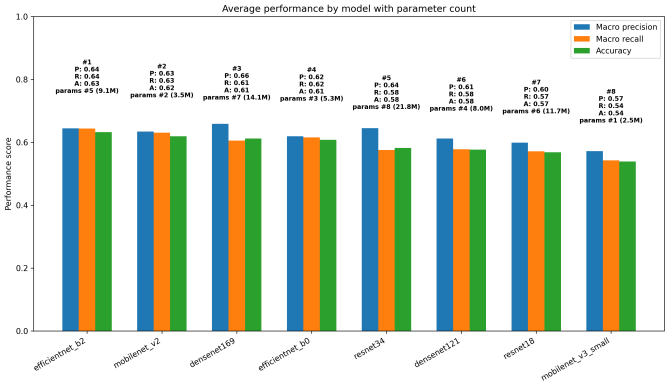

Fig. 10. Comparison of model families and sizes based on test accuracy, macro precision, and macro recall. Each bar represents the average performance across all training configurations for that model architecture. The larger models within each family tend to perform better than the smaller models within the same family, which suggests that larger models may be able to learn more robust features from the limited data available in the small-data regime. Figure also shows the parameter count for each model architecture, which is a common measure of model size.

C. Blue Class Difficulty and Class Imbalance

One interesting takeaway from the results is that performance appears to be closely tied to the ability of models to learn features that separate the Blue class from the Marten class. Given that most models will predict Marten for Blue images more than even the correct Blue class, it seems that this task is particularly difficult for the models in this experiment. This is despite the fact that both classes have a similar number of individuals and images in the dataset, which suggests that the difficulty is not simply due to class imbalance.

In fact, the place where class imbalance would be expected to have the most effect is on the Double Rex/Hairless class, which has only 17 individuals and 45 images in the dataset, even less so in the train split, yet it still shows acceptable precision and recall. This suggests that ambiguity and overlap between classes may be a more important factor than class imbalance in certain cases, and that even a small number of examples can be sufficient for learning robust features if the class is visually distinctive enough. However, it is important to note that this observation may be specific to this dataset and the particular classes involved, and it may not generalize to all small-data classification problems.

D. Limitations and Future Work

As mentioned, there are several limitations to this study that should be considered when interpreting the results. The first and most obvious limitation is the dataset itself. Although it is intentionally designed to be small with class imbalance, Section IV-A shows that models perform much better on some classes than others, and that overall performance appears to be closely tied to performance on a single difficult class (Blue). This limits how broadly the present results should be generalized. Future work could involve expanding the dataset to include more classes, more individuals per class, and more images per individual in order to create a more balanced and representative dataset for studying small-data image classification. It may also be worth exploring other small datasets in order to see if the trends observed here hold in other contexts, or if they are specific to this dataset and its particular characteristics.

Furthermore, the weak average effect of stronger augmentation may reflect the specific characteristics of this dataset and test split rather than a broad limitation of augmentation itself. Future work could involve testing the models on a more diverse and representative test set in order to better understand whether augmentation behaves differently under a wider target distribution.

Finally, there are many other design choices that we did not explore in this study that could affect performance in the small-data regime. For example, it may be worth exploring other training strategies such as cross-validation, class-balanced losses, or self-supervised pretraining. Additionally, it may be worth exploring other model architectures beyond the four families evaluated here. Overall, this study provides a narrow case study of how a few design choices behaved under one dataset, split procedure, and training recipe, and it does not settle the broader question of how best to approach small-data image classification.

VI. Conclusion

In this work, we conducted a small-scale exploration of how different design choices affected image classification performance under small-data constraints. We evaluated four model families with two size variants each, and we compared three training factors: learning rate, augmentation strength, and freezing schedule. Within this experiment, the choice of model architecture had a larger association with performance than the training-strategy differences, though that conclusion should be limited to these experiments. The lower learning rate was the strongest training-strategy effect, while augmentation strength and freezing schedule showed weaker effects. Furthermore, performance varied across classes, with the Blue class showing the weakest performance. These results suggest that a strong pretrained architecture and a less aggressive training strategy may be more effective than stronger augmentation, although these trends may be specific to this dataset and training recipe. Future work could involve expanding the dataset, testing on a less homogeneous test set, and exploring other design choices.

Acknowledgment

We gratefully acknowledge the breeders who gave permission for their rat images to be included in this dataset. These breeders include:

- Kolohe 'Iole Rattery - koloheiolerattery.com
- Little Paws Rattery - littlepawsrattery.weebly.com

We ask that you please consider supporting these breeders if you find this work useful, as they are the ones who made this dataset possible by sharing their photographs of their rats. The code used to run the experiments in this paper is available at https://github.com/NathanWithNoCathan/RatClass. The repository includes the code for the dataset splitting procedure, the training and evaluation pipelines, the code for generating the figures, and the dataset.

References

[1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[4] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[5] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, p. 60, 2019.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[7] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[8] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[9] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," *arXiv preprint arXiv:1708.04896*, 2017.

[10] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "Cutmix: Regularization strategy to train strong classifiers with localizable features," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[14] ——, "Sgdr: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations (ICLR)*, 2017.